Name- Najib Aftab
Roll- UG/04/BTCSE/2025/003
Reg no- AU/2025/0000139

1. Data and a pointer to the next node. (b)

2. Null (c)
3. Self-referential Structure (b)
4) head == null (b)
5) newnode->next = head; head = newnode (b)
6) temp = head; head = head->next free(temp) (b)
7) Traversing from tail to head (c)
8) n (b)
9) Name- Najib Aftab Roll- UG/04/BTCSE/2025/003
Reg-no- AU/2025/003

9) copying the data of p->next into p then deleting p->next
(c)
10) While (P1 = null) (b)
11) P->next == Null. (b)
12) Two pointers advancing at different rates (c)
13) Three (c)
14) Exactly one node (b)
15) P = (Struct node *) malloc (size of (Struct node));
16) Avoid special-case handelling for insertion and deletion at the head (b)
17) The first node (c)
18) It does not support random (indexed) access
19) Loss of access to all nodes causing a memory leak

Name- Najib Aftab Roll- UG/04/BTCSE/2025/003
Regno- AU/2025/0000139

Name - Najib Aftab
Roll- UG/104/BTCSE/2025/003
Regno - AU/2025/0000139

20. A slow pointer and a fast pointer (b)
21. the last node (c)
22. Nodes may be scattered anywhere in memory
23. Struct node {
    int data;
    struct node* next;
    };
24. Given.
    10->20->30->40-> NULL
the loop prints p->data and p->next->data
Output
10->20
20->30
30->40

~~25. Void insertEnd~~

Name - Najib Aftab
Roll- UG/104/BTCSE/2025/003

25) void insertEnd (struct node** head, int x)
{
struct node *t = (struct node *) malloc(sizeof(struct node));
t->data = x;
t->next = NULL
if (*head == NULL)
{
*head = t;
return;
struct node *p = *head;
while (p->next != NULL)
p = p->next
p->next = t;

Najib Aftab

~~26) Struct node* reverse~~

Majib Aftab Roll: UU1041BTCSE12025033

26) struct node* reverse (struct node *head)

```
{ struct node* prev = NULL;
  struct node* curr = head;
  struct node* next;
  while (curr != NULL)
  { next = curr->next;
    curr->next = prev;
    prev = curr;
    curr = next;
  }
  return prev;
}
```

Ex-) 10 -) 20 - 30 -> NULL

becomes 30 - 20 -> 10 -) NULL

27) Suppose the list is

10 -) 20 - 30 -) NULL

to delete 20:

Before

10 → 20 → 30

↑

Delete

we change the next pointer of b

temp->next = temp->next->next;

After

10 -> 30 - NULL

If temp points to the node before the node to be deleted:

struct node* del = temp->next

temp->next = del->next;

free(del)

Majib Aftab

28. Initial list

5 -> 8 -> 12 -> 3 - NULL

i) Insert 7 at the begining

ii) 7 - 5 -> 8 - 12 -> 3 -> NULL

iii) Delete the node containing 12

-> 5 -> 8 -> 3 - NULL

iii) Insert 9 at the end

-> 5 -> 8 -> 3 -> 9 -> NULL

Najib Aftab

Roll- U41041BTCSE/2025/003